Keywords: KV cache compression • mixture-of-experts • Fisher information • efficient LLM inference

MoEZip: Routing-Aware KV Cache Compression for Sparse Mixture-of-Experts LLMs

A Fisher-Information Routing Metric Cuts KV Memory by 10× While Preserving Expert Dispatch—Outperforming the Prior Art by 2.3–4.9×

By Minsang Kim, Seungjun Baek

arXiv:2610.30206 • NeurIPS 2026

Sparse mixture-of-experts (MoE) language models are uniquely vulnerable to KV cache eviction: removing context entries does not merely reduce attention coverage—it changes which experts are selected at each decode step, corrupting generation in ways that attention-only importance scores cannot predict. MoEZip closes this gap with a Fisher-information-based routing sensitivity metric that, combined with instruction-token attention weight, identifies which KV entries can be safely dropped. At a 10% retention ratio, MoEZip surpasses the prior state-of-the-art by +24 and +20 average points on LongBench and RULER—a 2.3× and 4.9× multiplicative improvement—without any model fine-tuning.

AT A GLANCE

Problem: KV eviction in sparse MoE LLMs alters which experts are selected at decode time; attention-only scores are blind to this routing perturbation.

Why it matters: MoE models (Mixtral, DeepSeek-MoE, etc.) are the dominant efficiency paradigm; deploying them at long contexts requires KV compression, yet existing methods degrade generation quality.

Method: Routing sensitivity via per-position Fisher information of the router distribution, combined with instruction-token attention scores, gives a dual-signal importance measure for each KV entry.

Result: At 10% KV retention, MoEZip scores 2.3× higher on LongBench and 4.9× higher on RULER than the prior SoTA (KVZip).

Evidence: Evaluation on LongBench, RULER, and NIAH across both softmax-type and sigmoid-type MoE routing architectures.

The Big Picture

Sparse MoE models reduce per-token floating-point operations by routing each token to a small subset of expert feed-forward networks. The key computational saving is that most parameters remain dormant on any given token. However, memory at inference scales with context length through the KV cache, and reducing it through eviction is essential for long-context deployment.

The standard approach—evict entries with the lowest cumulative attention weight—works well for dense transformers, where every parameter is active for every token. It fails silently in MoE models because the router is a function of the hidden state produced by the attention layer, and the hidden state depends on which KV entries are present. Removing a seemingly low-attention entry can shift the attention distribution enough to change the router's top-k expert selection, propagating an error that compounds across all subsequent layers.

Why Existing Approaches Fall Short

Attention-only scoring (H2O, SnapKV, PyramidKV): These methods score KV entries by the attention weight they accumulate across query tokens. They are designed for dense transformers and have no term that reflects the router's sensitivity to perturbations in the attention output.

Magnitude-based pruning: Low-magnitude entries may reside in positions where the routing gate function is highly sensitive; magnitude and routing sensitivity are not correlated in general.

KVZip: The prior SoTA for MoE KV compression, but its importance score does not account for routing distribution shift caused by eviction, leaving a systematic gap that MoEZip addresses.

Core Mechanism

MoEZip scores each KV entry (i, l) (context position i, layer l) with two signals:

1. **Routing sensitivity $\mathcal{F}_l(i)$:** How much does evicting position i shift the expert selection probabilities at layer l? Derived from the Fisher information of the routing distribution with respect to the attention output at position i.
2. **Instruction attention $\alpha_l(i)$:** The accumulated attention weight that instruction tokens (the task-relevant prefix) assign to position i at layer l.

The final importance score is $\phi_l(i) = \alpha_l(i) + \lambda \cdot \mathcal{F}_l(i)$, with λ balancing the two signals. Entries with the lowest $\phi_l(i)$ are evicted. No gradient computation or additional data is required.

Technical Formulation

Let $p_l(i) = \text{softmax}(W_r h_l(i))$ be the expert routing distribution at layer l, conditioned on the hidden state $h_l(i)$

produced by attending over context position i. The Fisher-information routing sensitivity is:

$$\mathcal{F}_l(i) = \mathbb{E}_{e\sim p_l(i)}\Big[\big(\nabla_{h_l(i)} \log p_l(i)[e]\big)^2\Big]$$

For softmax routing, this simplifies to a closed form involving the variance of the routing logits:

$$\mathcal{F}_l(i) = \mathrm{Var}_{e\sim p_l(i)}\Big[W_r^{(e)} h_l(i)\Big]$$

where $W_r^{(e)}$ is the e-th row of the routing weight matrix. An analogous closed form is derived for sigmoid-type routing. The combined score is:

$$\phi_l(i) = \alpha_l(i) + \lambda \cdot \mathcal{F}_l(i)$$

Entries with the lowest $\phi_l(i)$ values are evicted to meet the retention budget.

Learning or Inference Procedure

MoEZip is entirely training-free. At inference:

1. During the prefill phase, compute $\mathcal{F}_l(i)$ and $\alpha_l(i)$ for each KV position i and layer l.
2. Sort positions by $\phi_l(i)$ in descending order and retain the top-k (where k is the retention budget).
3. Proceed with autoregressive decoding over the compressed KV cache.

No gradient is computed; the Fisher computation reuses quantities already available from the attention forward pass. The method is plug-in compatible with any MoE LLM.

What the Guarantee Says

The Fisher-information derivation provides a local linear approximation to the change in the routing distribution caused by evicting position i. Specifically, the first-order routing perturbation $\Delta p_l(i)$ satisfies:

$$\|\Delta p_l(i)\|_2^2 \approx \mathcal{F}_l(i) \cdot \|\Delta h_l(i)\|_2^2$$

where $\Delta h_l(i)$ is the change in the hidden state caused by removing entry i. Retaining entries in decreasing order of $\mathcal{F}_l(i)$ therefore minimizes the worst-case routing perturbation norm at the given budget.

Experimental Findings

- **LongBench (long-document understanding):** At 10% KV retention, MoEZip scores 2.3× higher than KVZip on the average metric across six long-document subtasks.
- **RULER (structured reasoning):** MoEZip achieves 4.9× higher scores than KVZip at 10% retention; the gains are especially large on multi-hop retrieval subtasks where expert dispatch plays a critical role.
- **NIAH (needle-in-a-haystack):** MoEZip successfully retrieves needles at compression ratios where all attention-only baselines fail.
- **Both routing types:** Results are consistent across softmax-type (Mixtral-style) and sigmoid-type (DeepSeek-style) MoE architectures.
- **High retention convergence:** At $\geq$50% retention, all methods converge toward full-cache performance, confirming the gains are specific to the high-compression regime.

Ablations and Interpretation

Routing-only scoring ($\lambda = 1$, $\alpha = 0$): Underperforms on tasks where instruction-token attention is the dominant signal; confirms both terms are needed.

Attention-only scoring ($\lambda = 0$): Matches existing dense-transformer baselines, confirming that the routing-aware term alone drives the MoEZip gains.

Sensitivity to λ: Performance is robust to $\lambda \in [0.5, 2]$; the default $\lambda = 1$ performs well across all benchmarks.

Computational overhead: The Fisher computation adds $< 5\%$ latency to the prefill phase and is negligible relative to the decode speedup from the compressed KV cache.

Reference

Minsang Kim, Seungjun Baek. **MoEZip: Routing-Aware KV Cache Compression for Sparse Mixture-of-Experts LLMs**. arXiv:2610.30206. NeurIPS 2026. https://arxiv.org/abs/2610.30206